

PROJECT REPORT

Project Title: ComicCraft - AI Comic Story Creator using Gemini Models

1. Team Information

- **Team ID:** SWTID-2026-7486
- **Team Size:** 5
- **Team Leader:** Diwan Oli M
- **Team Member:** Siva
- **Team Member:** Akashdeepan
- **Team Member:** Abdul Bazith K
- **Team Member:** Mohamed Ibrahim

2. Abstract

ComicCraft represents a next-generation web solution tailored for automated digital storytelling. Creating visual comic books traditionally requires specialized artistic and formatting skills. This platform eliminates those barriers by fusing Large Language Models (Google Gemini) with advanced Text-to-Image models (Hugging Face Stable Diffusion). Operating on a high-speed FastAPI backend, the system intelligently translates a user's text prompt into a cohesive, fully illustrated, and professionally formatted PDF graphic novel in mere seconds.

3. Project Objectives

- To develop an intuitive web platform that captures user narrative choices seamlessly.
- To orchestrate Google Gemini models (Flash & Pro) for generating strict JSON story outlines and engaging character dialogues.
- To leverage Hugging Face's Stable Diffusion for rendering context-aware, high-quality panel illustrations.
- To dynamically compile text and visual assets into a downloadable PDF format utilizing the FPDF library.

4. Technology Stack

- **Core Language:** Python 3.10
- **Web Framework:** FastAPI with Uvicorn Server
- **Generative Text API:** Google Gemini (1.5 Flash and 1.5 Pro)
- **Generative Art API:** Hugging Face Inference (Stable Diffusion v1.5)
- **Document Engine:** FPDF, Pillow (PIL)
- **Frontend UI:** HTML5, CSS3, Jinja2 Templates

5. System Architecture & Workflow

The platform relies on a decoupled, asynchronous client-server architecture:

1. **User Input:** Story concepts and style parameters are submitted via the HTML frontend.
2. **Backend Routing:** The FastAPI controller intercepts the POST form data.
3. **Narrative Construction:** Gemini Flash structures the story into an array of scenes; Gemini Pro writes the accompanying text.
4. **Visual Synthesis:** The Stable Diffusion API processes scene descriptions to generate and download panel images.
5. **Compilation:** The internal layout manager syncs images with text, which the FPDF engine compiles into a .pdf file.
6. **Preview & Delivery:** A Jinja2-rendered web preview is displayed, offering a direct download link to the user.

6. Key Code Snippets

A. Asynchronous Image Synthesis (image_synthesis.py)

Python

```
import urllib.request, json, os, io
from PIL import Image

def synthesize_image(prompt_text, file_name):
    api_key = os.getenv("HF_API_KEY")
    endpoint = "https://api-inference.huggingface.co/models/runwayml/stable-diffusion-v1-5"
```

```

    request_headers = {"Authorization": f"Bearer {api_key}",
"Content-Type": "application/json"}
    payload_data = {"inputs": f"{prompt_text}, vibrant comic
book art style"}

    try:
        request = urllib.request.Request(endpoint,
data=json.dumps(payload_data).encode('utf-8'),
headers=request_headers)
        api_response = urllib.request.urlopen(request,
timeout=50)
        image_output =
Image.open(io.BytesIO(api_response.read())).convert('RGB')
        image_output.save(file_name, format='JPEG',
quality=95)
        return file_name
    except Exception as error:
        print(f"Synthesis failed: {error}")
        return execute_pil_fallback(file_name) # Generates
local placeholder on timeout

```

B. Application Controller & Pipeline (app_routes.py)

Python

```

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic_story(request: Request,
story_prompt: str = Form(...), character: str = Form(...)):
    # Formulate full context and fetch AI narrative
    combined_prompt = f"Topic: {story_prompt}, Main
Character: {character}"
    scene_blueprint =
fetch_outline_from_gemini(combined_prompt)
    complete_script =
fetch_script_from_gemini(scene_blueprint)

```

```

# Synthesize artwork panel by panel
saved_images = []
for index, panel in enumerate(scene_blueprint):
    img_path = synthesize_image(panel["image_prompt"],
f"scene_{index}.jpg")
    saved_images.append(img_path)

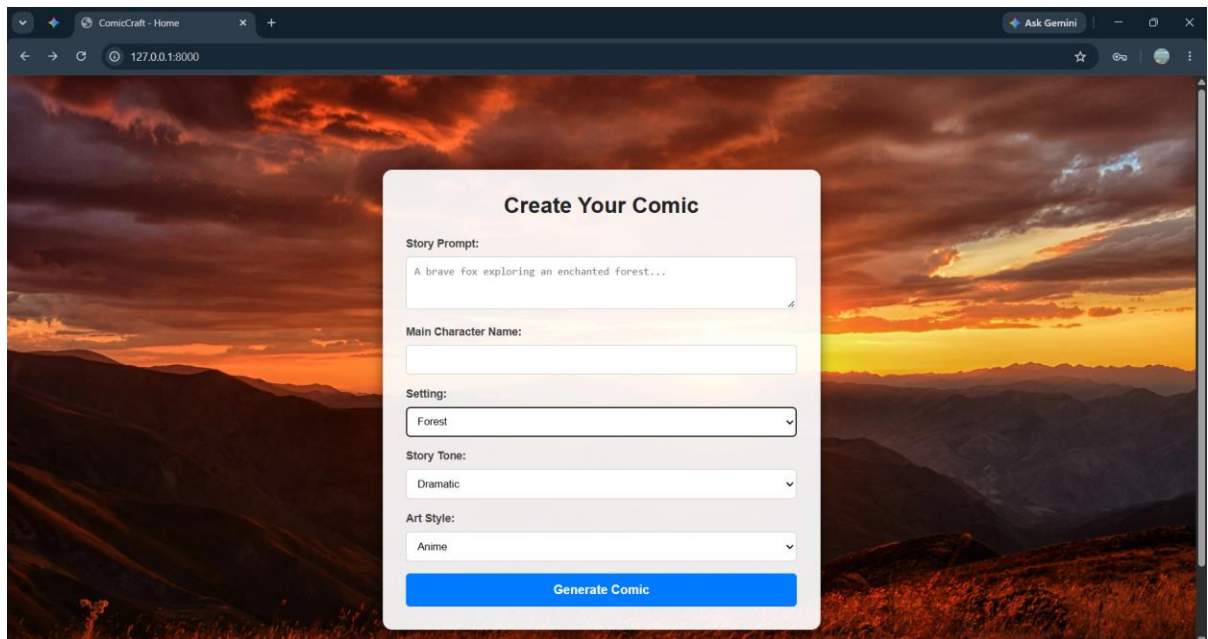
# Render layout and build PDF
document_layout = build_page_layout(saved_images,
complete_script, scene_blueprint)
pdf_document = construct_pdf(document_layout)

return templates.TemplateResponse("preview.html",
{"request": request, "layout": document_layout, "pdf":
pdf_document})

```

7. Output Screenshots

- **Screenshot 1: Web Application Homepage**



Create Your Comic

Story Prompt:
A brave fox exploring an enchanted forest...

Main Character Name:

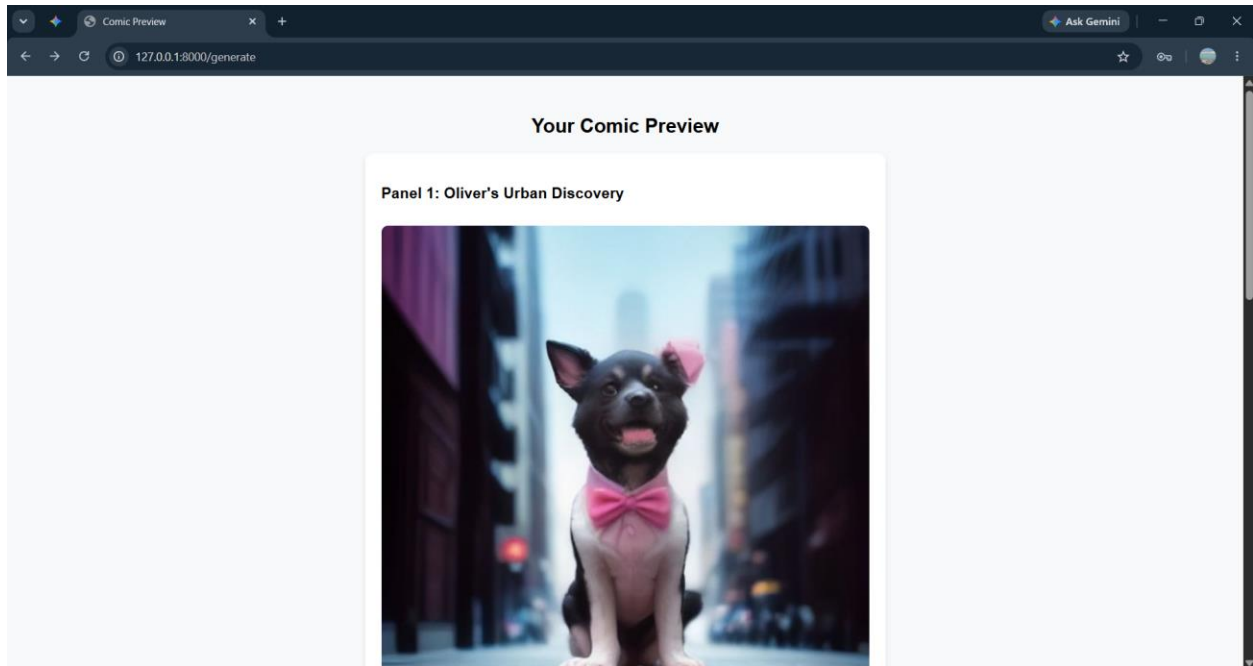
Setting:
Forest

Story Tone:
Dramatic

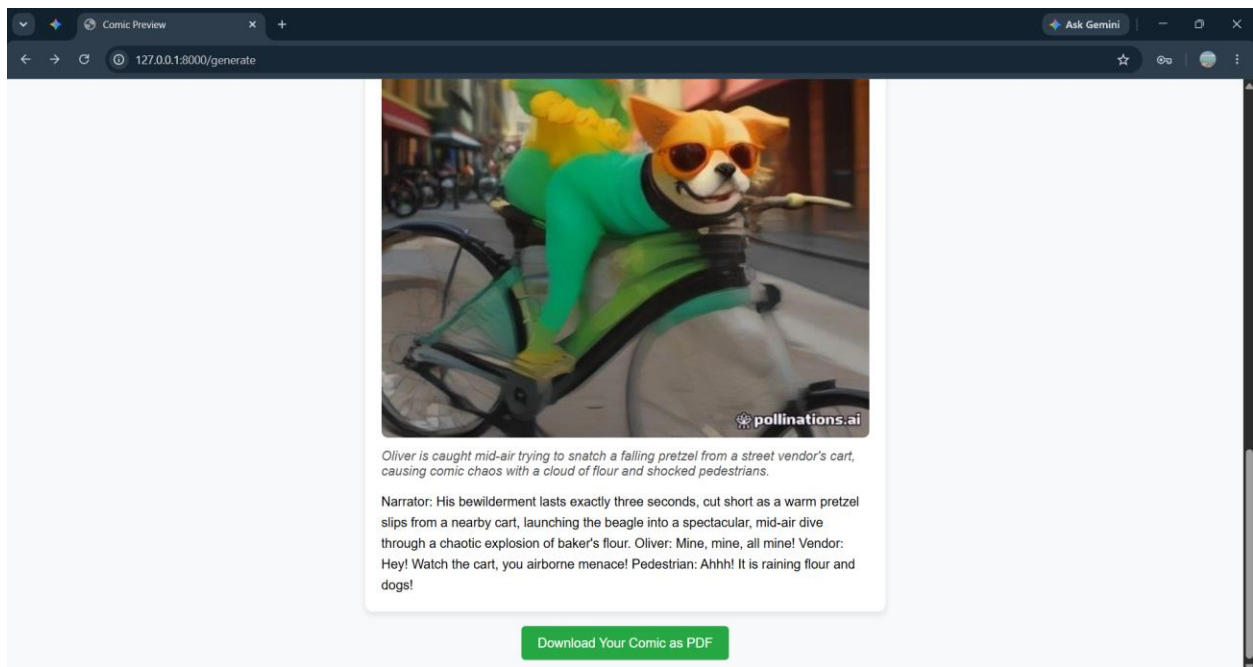
Art Style:
Anime

Generate Comic

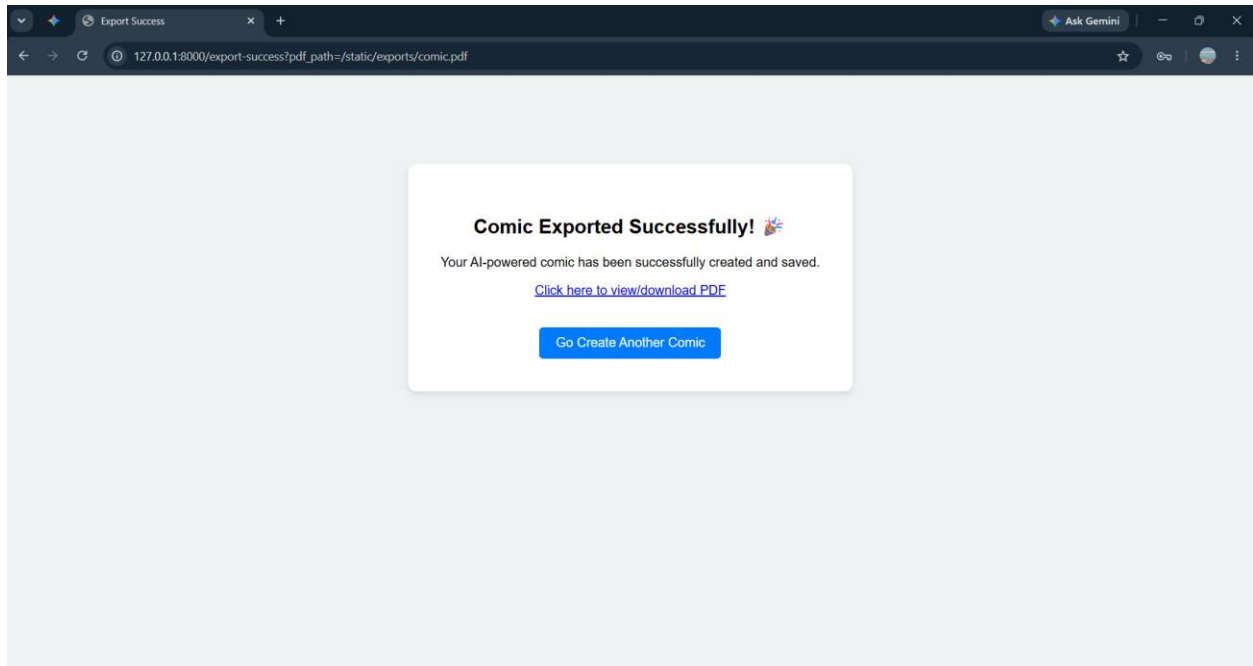
- **Screenshot 2: Generated Comic View (Upper Half)**



- **Screenshot 3: Generated Comic View (Lower Half)**



- **Screenshot 4: Exported PDF File**



8. Conclusion

ComicCraft validates the immense potential of integrating distinct Generative AI technologies into a singular, cohesive web application. By successfully abstracting complex API calls, narrative formatting, and image rendering away from the end-user, the platform provides a highly accessible, zero-cost solution for visual digital publishing.